Task 1 — QA & Debug Report

Prepared by Nikhil · Application: TodoMVC (React) · https://todomvc.com/examples/react/dist/

1. Summary for a non-technical stakeholder

The app handles the basics (add, complete, edit with Enter, delete with a mouse, filter) but is **not production ready**. The biggest problems: **everything the user types disappears when the page is refreshed**, and the **delete button cannot be used on a phone, tablet or with a keyboard**. There are also several accessibility gaps and small polish issues. 8 issues are documented below: 2 High, 3 Medium, 3 Low.

2. Scope & method

App	TodoMVC React example (live). The RealWorld public demo/API was tried first but is offline (demo.realworld.io returns 404; api.realworld.io returns Cloudflare error 1016), so I switched to TodoMVC.
Environment	macOS, Chromium 158 (Playwright, headless) plus manual checks; desktop 1000×800 and mobile 320/390 px; touch emulation.
Flows exercised	add, complete, edit (Enter / Esc / blur), delete, clear completed, toggle-all, filters (All / Active / Completed), refresh, empty & whitespace input, very long input, HTML/script input, keyboard-only navigation, mobile layout.
Passed	Empty and whitespace-only input rejected · leading/trailing spaces trimmed · `<img onerror>` input rendered as text, no XSS · editing to empty text removes the item (expected TodoMVC behaviour) · blur saves an edit · counter, filters and clear-completed correct · no horizontal scroll at 320 px · long text wraps without breaking the layout.

3. Bug table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	**Todos are lost on page refresh**	1. Open the app 2. Add 2 todos 3. Press F5 / reload	**Expected:** todos are still there. **Actual:** list is empty; localStorage has no keys.	**High**	State lives only in React memory; the shipped bundle contains zero references to localStorage / any persistence layer.
2	**Escape does not cancel an edit**	1. Add a todo 2. Double-click it to edit 3. Change the text, press Esc	**Expected:** edit is cancelled, original text restored. **Actual:** item stays in edit mode with the changed text still in the box (see Fig. 1).	**Medium**	Key handler for Esc is missing/ignored in the edit input; only Enter and blur are handled.
3	**Delete button is unreachable on touch devices and by keyboard**	1. Add a todo 2. On a touch/mobile viewport (no hover), look for the delete (x) button 3. Or press Tab through the page	**Expected:** a visible, focusable delete control. **Actual:** button is `display:none` until mouse hover; Tab order skips it entirely (toggle-all → toggle → filters).	**High**	Delete button is revealed only via CSS :hover; display:none removes it from the tab order and touch users never trigger hover.

4	**Todo text cannot be edited with the keyboard**	1. Add a todo 2. Tab to the todo label and try Enter	**Expected:** a keyboard way to enter edit mode. **Actual:** the label has tabindex -1; editing only works via double-click.	**Medium**	Edit mode is bound to the onDoubleClick event only; no keyboard equivalent.
5	**Completion checkbox has no accessible name**	1. Add a todo 2. Inspect the checkbox (or use a screen reader)	**Expected:** announced as "Mark <todo text> complete". **Actual:** no aria-label, no associated <label>; screen readers announce an unnamed checkbox.	**Medium**	Checkbox rendered with no label association (aria-label null, labels.length = 0).
6	**No length limit on todo text**	1. Paste a 5,000-character string into the input, press Enter	**Expected:** a sensible limit or a validation message. **Actual:** accepted silently; a 300-char word fills ~9 lines of the list (Fig. 2).	**Low**	No maxLength / validation on the new-todo input.
7	**Counter text reads "1 item left!"**	1. Add one todo 2. Read the footer counter	**Expected:** "1 item left" (neutral wording). **Actual:** "1 item left!" / "2 items left!" with an exclamation mark.	**Low**	Hard-coded string in the footer component.
8	**Third-party trackers loaded with no notice; no security headers**	1. Open the page source / DevTools Network 2. Check response headers	**Expected:** consent notice for analytics; CSP and X-Content-Type-Options headers. **Actual:** Google Tag Manager and Analytics load on page view; no CSP / X-Frame-Options / X-Content-Type-Options header.	**Low**	Analytics snippet in index.html; static hosting on GitHub Pages does not send custom security headers.

Evidence

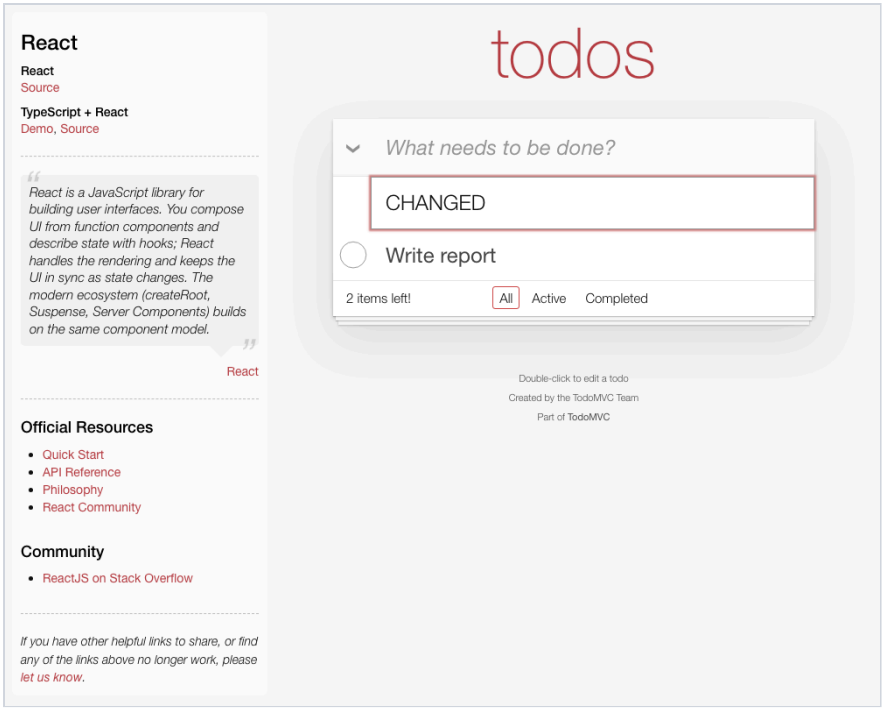


Fig. 1 — Bug 2: after pressing Esc the edit box is still open and still shows "CHANGED".

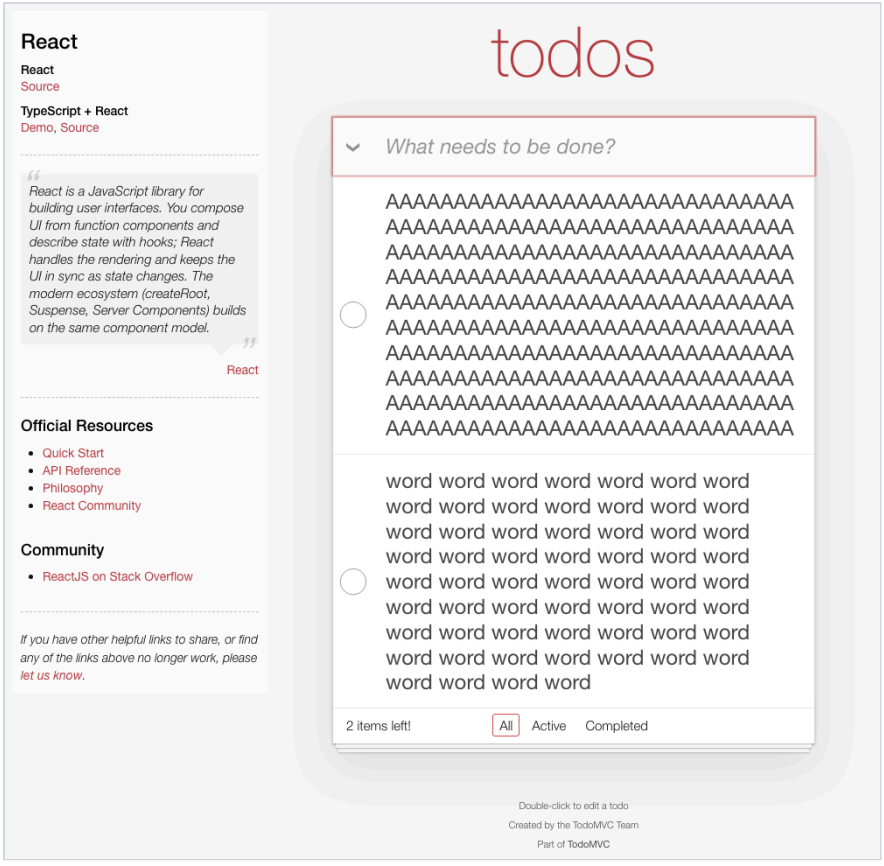


Fig. 2 — Bug 6: a 300-character string (and a 60-word string) accepted with no limit.

4. Root-cause analysis — Bug 1 (data lost on refresh)

What is happening: every todo disappears when the page is reloaded; after adding two todos and reloading, the list is empty and `localStorage` contains no keys. **Why:** the todo list lives only in React component state (a reducer held in memory). When the page reloads, the JavaScript runtime is destroyed and the state is re-initialised to an empty array. I confirmed this by downloading the shipped `app.bundle.js` (237 KB) and searching it: it contains zero references to `localStorage`, `sessionStorage`, IndexedDB or any network call, so there is no persistence layer at all. **Impact:** users lose all of their data on an accidental refresh, tab close or browser restart, which defeats the purpose of a to-do app. **Fix:** (1) initialise the reducer state lazily from `localStorage` (`useReducer(reducer, null, () => JSON.parse(localStorage.getItem('todos') || '[]'))`), wrapped in try/catch for corrupt or disabled storage; (2) add a `useEffect` that writes the todos back to `localStorage` whenever they change; (3) for multi-device use, move to a backend API; (4) add a regression test that adds a todo, reloads and asserts the todo is still present.